

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO5: Analyze object-oriented software using quality assurance and usability testing Techniques (BL4, BL5)
Assessment Tool Weightage: 25%

QUESTIONS: 05

Total Marks:25

Annexure A

Error Analysis Task

Code of Conduct:

*I E. Sakthi priya (192520041) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

E. Sakthi priya

RUBRICS FOR CALCULATION:

Error Analysis Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Annexure A

Error Analysis Task

Q1. Scenario: Online Banking System

Modules:

User Login, Fund Transfer, Account Verification

Question:

Write pseudocode for a quality assurance testing process that validates secure user authentication and fund transfer operations. Include input validation, transaction verification, exception handling, and test result logging.

Q2. Scenario: E-Commerce Shopping Platform

Features:

Product Search, Shopping Cart, Payment Gateway

Question:

Design pseudocode for a usability testing evaluation that measures customer interaction efficiency, including product search time, checkout completion time, and navigation complexity. Explain how the algorithm collects and analyzes user behavior data.

Q3. Scenario: Hospital Management System

Modules:

Patient Registration, Appointment Scheduling, Medical Records

Question:

Write pseudocode to generate and execute test cases for appointment scheduling functionality. Include validation of doctor availability, appointment conflicts, patient information accuracy, and system responses to invalid requests.

Q4. Scenario: Airline Reservation System

Modules:

Passenger Registration, Flight Booking, Seat Allocation

Question:

Develop pseudocode for a constraint-based testing approach where booking requests must satisfy conditions such as valid passenger information, seat availability, and payment confirmation. Show how invalid booking scenarios are detected and handled.

Q5. Scenario: Smart Library Management System

Features:

Book Search, Book Reservation, Borrow/Return Services

Question:

Write pseudocode for integrating quality assurance and usability testing where the system evaluates both functional correctness (reservation and borrowing operations) and user experience factors such as response time, ease of navigation, and accessibility.

Q1. Online Banking System

The system contains User Login, Fund Transfer, and Account Verification. The pseudocode must include input validation, transaction verification, exception handling, and test-result logging.

PSEUDOCODE :

BEGIN

Initialize TestLog

DISPLAY "Enter username and password"

INPUT username, password

IF username is empty OR password is empty THEN

LOG "FAIL: Empty login credentials"

DISPLAY "Invalid input"

STOP

END IF

TRY

result ← AuthenticateUser(username, password)

IF result = FALSE THEN

LOG "FAIL: Authentication unsuccessful"

DISPLAY "Invalid username or password"

STOP

END IF

LOG "PASS: User authenticated successfully"

DISPLAY "Enter source account, destination account and amount"

INPUT sourceAccount, destinationAccount, amount

IF sourceAccount is empty OR destinationAccount is empty THEN

LOG "FAIL: Invalid account details"

DISPLAY "Invalid account information"

STOP

END IF

IF amount ≤ 0 THEN

LOG "FAIL: Invalid transfer amount"

DISPLAY "Amount must be greater than zero"

STOP

END IF

verified ← VerifyAccount(destinationAccount)

```

IF verified = FALSE THEN
    LOG "FAIL: Destination account verification failed"
    DISPLAY "Invalid destination account"
    STOP
END IF

balance ← GetBalance(sourceAccount)

IF balance < amount THEN
    LOG "FAIL: Insufficient balance"
    DISPLAY "Insufficient funds"
    STOP
END IF

transaction ← TransferFunds(
    sourceAccount,
    destinationAccount,
    amount)

IF transaction = SUCCESS THEN
    LOG "PASS: Fund transfer completed"
    DISPLAY "Transfer successful"
ELSE
    LOG "FAIL: Fund transfer unsuccessful"
    DISPLAY "Transfer failed"
END IF

CATCH Exception
    LOG "ERROR: Transaction exception occurred"
    DISPLAY "Transaction could not be completed"
END TRY

Save TestLog

END

```

- **Input validation** prevents empty or invalid account and amount values.
- **Authentication testing** verifies secure login.
- **Account verification** ensures that funds are sent to a valid account.
- **Transaction verification** checks whether the transfer was successful.
- **Exception handling** prevents unexpected failures from crashing the system.
- **Test logging** records PASS, FAIL, and ERROR results for quality analysis.

Q2. E-Commerce Shopping Platform

The system includes Product Search, Shopping Cart, and Payment Gateway. The usability evaluation must measure product search time, checkout completion time, and navigation complexity.

PSEUDOCODE :

BEGIN

Initialize UserResults

FOR each test user

START SearchTimer

DISPLAY "Search for a product"

INPUT searchKeyword

result $\leftarrow$ SearchProduct(searchKeyword)

STOP SearchTimer

searchTime $\leftarrow$ GetElapsedTime(SearchTimer)

IF result is empty THEN

record searchSuccess = FALSE

ELSE

record searchSuccess = TRUE

END IF

navigationSteps $\leftarrow$ 0

WHILE user has not completed checkout

DISPLAY current page

navigationSteps $\leftarrow$ navigationSteps + 1

IF user selects "Add to Cart" THEN

AddProductToCart()

END IF

IF user selects "Checkout" THEN

START CheckoutTimer

ProcessCheckout()

STOP CheckoutTimer

checkoutTime $\leftarrow$ GetElapsedTime(CheckoutTimer)

BREAK

END IF

```

END WHILE

Store(
    searchTime,
    checkoutTime,
    navigationSteps,
    searchSuccess
)

END FOR

averageSearchTime ← CalculateAverage(searchTime)
averageCheckoutTime ← CalculateAverage(checkoutTime)
averageNavigationSteps ← CalculateAverage(navigationSteps)

DISPLAY "Average Search Time:", averageSearchTime
DISPLAY "Average Checkout Time:", averageCheckoutTime
DISPLAY "Average Navigation Steps:", averageNavigationSteps

IF averageSearchTime is high THEN
    DISPLAY "Improve product search"
END IF

IF averageCheckoutTime is high THEN
    DISPLAY "Simplify checkout process"
END IF

IF averageNavigationSteps is high THEN
    DISPLAY "Improve navigation design"
END IF

END

```

The algorithm records user behaviour while performing shopping tasks. It measures:

- **Product search time** – how quickly users find products.
- **Checkout completion time** – how long users take to complete checkout.
- **Navigation steps** – number of interactions needed to finish the task.
- **Search success** – whether the user successfully finds the required product.

The collected data is averaged across users. High values indicate areas where the interface should be improved.

Q3. Hospital Management System

The system contains Patient Registration, Appointment Scheduling, and Medical Records. The pseudocode must validate doctor availability, appointment conflicts, patient information and invalid requests.

PSEUDOCODE :

BEGIN

```
TestCases ← [  
    ValidAppointment,  
    InvalidPatient,  
    DoctorUnavailable,  
    AppointmentConflict,  
    InvalidDate  
]
```

FOR each TestCase in TestCases

```
    DISPLAY "Executing:", TestCase
```

```
    INPUT patientID  
    INPUT doctorID  
    INPUT appointmentDate  
    INPUT appointmentTime
```

```
    IF patientID is empty OR doctorID is empty THEN  
        DISPLAY "FAIL: Missing patient or doctor information"  
        CONTINUE  
    END IF
```

```
    patient ← FindPatient(patientID)
```

```
    IF patient does not exist THEN  
        DISPLAY "FAIL: Patient information is invalid"  
        CONTINUE  
    END IF
```

```
    IF appointmentDate is invalid THEN  
        DISPLAY "FAIL: Invalid appointment date"  
        CONTINUE  
    END IF
```

```
    available ← CheckDoctorAvailability(  
        doctorID,  
        appointmentDate,  
        appointmentTime)
```

```
    IF available = FALSE THEN  
        DISPLAY "FAIL: Doctor is unavailable"  
        CONTINUE  
    END IF
```

```
    conflict ← CheckAppointmentConflict(  
        doctorID,
```

```

        appointmentDate,
        appointmentTime)

    IF conflict = TRUE THEN
        DISPLAY "FAIL: Appointment conflict detected"
        CONTINUE
    END IF

    result ← ScheduleAppointment(
        patientID,
        doctorID,
        appointmentDate,
        appointmentTime)

    IF result = SUCCESS THEN
        DISPLAY "PASS: Appointment scheduled"
    ELSE
        DISPLAY "FAIL: Appointment could not be scheduled"
    END IF

END FOR

END

```

The test cases cover both valid and invalid situations.

- **Valid appointment:** should be successfully scheduled.
- **Invalid patient:** request should be rejected.
- **Doctor unavailable:** appointment should not be created.
- **Appointment conflict:** duplicate time slots should be prevented.
- **Invalid date:** system should display an appropriate error.

This ensures that appointment scheduling behaves correctly under different conditions.

Q4. Airline Reservation System

The system consists of Passenger Registration, Flight Booking, and Seat Allocation. A booking is valid only when passenger information, seat availability, and payment confirmation satisfy the required constraints.

PSEUDOCODE :

```

BEGIN

    INPUT passengerName
    INPUT passengerID
    INPUT flightID
    INPUT seatNumber
    INPUT paymentDetails

    IF passengerName is empty OR passengerID is empty THEN

```

```
    DISPLAY "INVALID: Passenger information"
    LOG "Booking rejected - invalid passenger"
    STOP
END IF
```

```
IF NOT ValidPassenger(passengerID) THEN
    DISPLAY "INVALID: Passenger details"
    LOG "Booking rejected - invalid passenger"
    STOP
END IF
```

```
IF NOT FlightExists(flightID) THEN
    DISPLAY "INVALID: Flight does not exist"
    LOG "Booking rejected - invalid flight"
    STOP
END IF
```

```
IF NOT SeatExists(flightID, seatNumber) THEN
    DISPLAY "INVALID: Seat does not exist"
    LOG "Booking rejected - invalid seat"
    STOP
END IF
```

```
IF NOT IsSeatAvailable(flightID, seatNumber) THEN
    DISPLAY "INVALID: Seat already booked"
    LOG "Booking rejected - seat unavailable"
    STOP
END IF
```

```
paymentStatus ← ProcessPayment(paymentDetails)
```

```
IF paymentStatus = FAILED THEN
    DISPLAY "INVALID: Payment failed"
    LOG "Booking rejected - payment failure"
    STOP
END IF
```

```
bookingStatus ← CreateBooking(
    passengerID,
    flightID,
    seatNumber)
```

```
IF bookingStatus = SUCCESS THEN
    DISPLAY "Booking confirmed"
    LOG "PASS: Booking successful"
ELSE
    DISPLAY "Booking failed"
    LOG "FAIL: Booking creation failed"
END IF
```

EXCEPTION

 DISPLAY "System error. Booking could not be completed."

 LOG "ERROR: Unexpected booking exception"

END

The algorithm follows a constraint-based approach:

Passenger valid → Flight valid → Seat available → Payment confirmed → Booking created

If any condition fails, the booking is rejected and the reason is recorded. This prevents invalid reservations and reduces booking errors.

Q5. Smart Library Management System

The system includes Book Search, Book Reservation, and Borrow/Return Services. The question requires both functional correctness and user-experience evaluation, including response time, navigation and accessibility.

PSEUDOCODE :

BEGIN

 Initialize TestResults

 FOR each test user

 // Functional Testing - Book Search

 START SearchTimer

 INPUT bookName

 result ← SearchBook(bookName)

 STOP SearchTimer

 searchTime ← GetElapsedTime(SearchTimer)

 IF result is not empty THEN

 LOG "PASS: Book search successful"

 ELSE

 LOG "FAIL: Book search unsuccessful"

 END IF

 // Usability Testing

 navigationSteps ← CountNavigationSteps()

 IF navigationSteps ≤ ExpectedSteps THEN

 LOG "PASS: Navigation is simple"

 ELSE

 LOG "FAIL: Navigation is complex"

```

END IF

// Reservation Testing
IF BookAvailable(bookName) THEN

    reservation ← ReserveBook(bookName)

    IF reservation = SUCCESS THEN
        LOG "PASS: Book reservation successful"
    ELSE
        LOG "FAIL: Book reservation failed"
    END IF

ELSE
    DISPLAY "Book unavailable"

    LOG "PASS: Unavailable book handled correctly"
END IF

// Borrow Testing
borrowResult ← BorrowBook(bookName)

IF borrowResult = SUCCESS THEN
    LOG "PASS: Borrow operation successful"
ELSE
    LOG "FAIL: Borrow operation failed"
END IF

// Accessibility Testing
accessibility ← CheckAccessibility()

IF accessibility = TRUE THEN
    LOG "PASS: Interface accessible"
ELSE
    LOG "FAIL: Accessibility issues detected"
END IF

Store(
    searchTime,
    navigationSteps,
    accessibility
)

END FOR

averageSearchTime ← CalculateAverage(searchTime)
averageNavigation ← CalculateAverage(navigationSteps)

DISPLAY "Average Search Time:", averageSearchTime
DISPLAY "Average Navigation Steps:", averageNavigation

```

DISPLAY "Accessibility Results"

END

The algorithm combines QA testing and usability testing.

Functional correctness is checked through:

- Book search.
- Book reservation.
- Borrow operation.
- Handling unavailable books.

User experience is evaluated through:

- Search response time.
- Number of navigation steps.
- Accessibility testing.

The collected results help identify both functional defects and usability problems.